

CONTEXTO: sistema-granados, para continuar en un chat nuevo

Soy Angel ("bro"). Este es mi proyecto personal, sistema web para la Municipalidad de Granados, Baja Verapaz. En este chat nuevo quiero avanzar en el módulo de disponibilidad presupuestaria.

Tengo también un proyecto de la universidad (sistema de quejas para otra municipalidad, aparte) que en algún momento futuro quiero fusionar con este en un solo producto, pero por ahora son independientes.

Qué es el sistema

Sistema web para la DAFIM (Dirección de Administración Financiera Integrada Municipal) y RRHH de la Municipalidad de Granados. Porteo de un Colab de Python a Java, calibrado con archivos reales de la municipalidad (no solo datos de prueba).

Stack tecnológico

- **Backend:** Spring Boot 3.2.5, Java 17
- **Vistas:** Thymeleaf (server-side, sin frontend framework)
- **Base de datos:** PostgreSQL (migrado desde MySQL en este mismo proyecto; ver detalles de la migración más abajo)
- **ORM:** Spring Data JPA / Hibernate, `spring.jpa.hibernate.ddl-auto=update` (nunca `create`/`create-drop`, para no perder datos)
- **Seguridad:** Spring Security con roles (SUPERADMIN, ADMIN_DAFIM, ADMIN_RRHH), rutas protegidas por prefijo (`/dafim/**`, `/rrhh/**`, `/admin/**`). Headers configurados con `frameOptions.sameOrigin()` para permitir vistas previas de PDF en iframe propio.
- **Librerías de parsing:** Apache POI 4.0.1 (Excel), PDFBox 2.0.29 (texto y merge de PDFs)
- **CSS propio:** sin frameworks (Bootstrap solo para íconos `bi-*`), variables CSS (`--pino`, `--jade`, `--papel`, etc), tema dorado/bronce oscuro inspirado en el escudo municipal. Archivo: `static/css/app.css`.
- **Empaquetado:** `spring-boot-maven-plugin`, fat-jar, `java -jar sistema-granados.jar`, puerto 8085

Estructura de paquetes Java

```
com.granados.sistema
├── usuarios/           (usuarios y roles)
├── config/             SecurityConfig, DataInitializer, StorageProperties
├── web/                DashboardController
├── dafim/
│   ├── compras/       Compras Directas (portado de Python)
│   └── paquetes/       Paquetes de facturas SAT
└── rrhh/               Recursos Humanos
```

Módulo 1: Compras Directas

Informe mensual de compras directas a partir de PDF SICOIN, reporte de cheques (.xls), bloque TXT de Guatecompras y PDFs de confirmación NPG. Calibrado con junio 2026: 543 filas, total Q2,951,154.59 cuadrado al centavo. No lo he usado activamente aún (estuve enfocado en paquetes); el trabajo de compras directas de estos meses ya está hecho aparte en un Colab en Drive.

Módulo 2: Paquetes de facturas SAT

La oficina manda un Excel mensual (cada hoja = un paquete) y por separado los PDFs de facturas FEL de la SAT. El sistema casa cada línea con su factura por monto exacto + similitud Jaccard de texto, nunca repite una factura (autorización UNIQUE), guarda todo por mes, e imprime cada paquete como un solo PDF en el orden del Excel. Validado con 183 facturas de junio (183/183) y 214 de julio (211 automáticas, 3 pendientes explicadas).

Mejoras visuales recién agregadas (este módulo es el más nuevo y activo):

- Buscador global por DTE (`GET /dafim/paquetes/buscar?dte=...`), busca en todos los meses guardados, muestra a qué paquete pertenece cada factura encontrada con link directo.
- Filtro rápido dentro de la vista de un mes: campo de texto que filtra en vivo por DTE o descripción entre todas las líneas de todos los paquetes del mes.
- Vista de un mes convertida en catálogo de tarjetas (`<details>` expandibles) en vez de scroll largo con todas las tablas apiladas. Clases CSS nuevas: `.catalogo-paquetes`, `.paquete-detalle`, `.paquete-cabecera`, `.filtro-rapido`, `.buscador-caja`.
- Vista previa de PDF en modal con cabecera mostrando DTE y emisor.

Módulo 3: Recursos Humanos

Personal (alta de empleados), Permisos (por días/horas, con/sin goce, aprobación, filtros combinables), adjunto de permiso físico escaneado (blob aparte), constancia formal imprimible con firma configurable.

Pendientes abiertos (el que sigue: presupuesto)

- **Módulo de disponibilidad presupuestaria** (el que voy a trabajar en el chat nuevo): cargar reporte de saldos SICOIN por estructura/renglón/fuente, pantalla "¿puedo pagar esta factura?" con sugerencia de renglón, semáforo presupuesto + dinero real en banco. Pendiente que yo mande un reporte real de saldos para calibrar.
- Módulo de remuneraciones auto-generadas: pendiente que yo envíe el archivo/Colab de referencia.
- Servidor local Windows en LAN de la municipalidad (guía completa ya en `servidor/LEEME_SERVIDOR.md`): IP fija, firewall puerto 8085, servicio con NSSM, respaldo diario con `pg_dump` (antes era `mysqldump`), hay que actualizar esa parte de la guía a Postgres).

Migración de MySQL a PostgreSQL (ya hecha en este proyecto)

Decidí migrar porque no había datos relevantes en la BD todavía (solo usé paquetes; compras directas ni se ha usado, ese trabajo ya está aparte en Drive), y porque el sistema de quejas de la universidad, con el que en algún momento quiero fusionar esto, ya está en Postgres.

Cambios hechos:

- `pom.xml`: driver `mysql-connector-j` → `postgresql`
- `application.properties`: URL, driver-class-name, usuario `postgres`, dialecto `PostgreSQLDialect`. Ojo: Postgres NO crea la base de datos sola (a diferencia de MySQL con `createDatabaseIfNotExist`), hay que crearla a mano una vez.
- `FacturaPdf.java` y `PermisoAdjunto.java`: `columnDefinition = "LONGBLOB"` → `"BYTEA"`, y se agregó `@JdbcTypeCode(Types.BINARY)` junto a `@Lob` en el campo `byte[] datos` (bug conocido de Hibernate 6 + Postgres: sin esa anotación, Hibernate trata el `@Lob` como Large Object/OID en vez de binario directo, y truena con "la columna es bytea pero la expresión es bigint").
- `SecurityConfig.java`: se agregó `.headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))` porque el header por defecto `X-Frame-Options: DENY` bloqueaba la vista previa de PDF en iframe (incluso siendo del mismo sitio).

Mi método de trabajo

Pruebo todo con archivos reales de la municipalidad antes de dar por bueno un módulo. Cuando algo falla, la respuesta correcta casi siempre es diagnosticar con los datos reales antes de tocar código a ciegas.

Prefiero ediciones puntuales (`str_replace`) sobre archivos existentes en vez de reescribir clases completas, para no gastar tokens, salvo cuando es un rediseño real de vistas (ahí sí toca reescribir el HTML completo).

No uso el guion largo (—) en las respuestas.